

StayBoost

Phase 2 Audit Report

Operational MVP - Reservation Data Spine

Date: 2026-06-16

Branch: claude/stayboost-product-blueprint-k3w0jp

Scope: full repository, read-only.

Critical framing

Phase 2 work to date is code-complete and CI-verified, but NOT yet provisioned or deployed. No cloud resources exist and nothing runs in production. This report distinguishes built/verified from live throughout.

Phase 1: 100% | Phase 2 (Operational MVP): ~25-30%

1. Infrastructure Status

Item	Built	Live	Notes
PostgreSQL + pgvector	[OK]	[X] not provisioned	compose + Railway config; deployment.md S1
Redis	[OK]	[X] not provisioned	required in prod (rate-limit store)
Migrations applied	[OK] 0001-0005	[~] CI only	migrate:deploy auto on release
RLS verified	[OK]	[~] CI only	automated cross-tenant isolation test

RLS detail (the P0 guarantee)

reservations and channel_connections use ENABLE + FORCE ROW LEVEL SECURITY with policies keyed on current_setting('app.org_id'). The API routes tenant work through PrismaService.withTenant(orgId, fn) (parameterized set_config). CI provisions Postgres+pgvector, migrates, and runs rls.integration.spec.ts proving a tenant cannot read or write another tenant's rows.

Production caveat: for FORCE RLS to fully bind, the API must connect as a NON-OWNER DB role. Verify this during provisioning (flagged in the runbook).

2. Deployment Status

Nothing is deployed. Deploy artifacts are complete and committed:

Target	Artifact	Status
Web -> Vercel	apps/web/vercel.json	[OK] ready / [X] not deployed
API -> Railway	Dockerfile, railway.json, .dockerignore	[OK] ready / [X] not deployed
AI -> Railway	Dockerfile, railway.json	[OK] ready / [X] not deployed

Environment variables

The full contract is defined and validated (Zod in config/env.ts, fail-fast in prod) and documented in .env.example + docs/deployment.md. NONE are set in any live environment because none exists. Required before first deploy: DATABASE_URL, REDIS_URL, JWT_SECRET, AUTH_COOKIE_SECURE=true, APP_URL, WEB_ORIGIN, COOKIE_DOMAIN, AI_SERVICE_URL, AI_SERVICE_TOKEN, RESEND_API_KEY, GOOGLE_*, ANTHROPIC_API_KEY, NEXT_PUBLIC_*.

3. Reservation Module [OK] built

- Models: Reservation (org-scoped, unique per (org, source, externalId), RLS) and ChannelConnection (iCal feed per property, RLS); canonical Zod contracts in @stayboost/domain.
- APIs: POST/GET /v1/channels, POST /v1/channels/:id/sync, GET /v1/reservations - JWT + RBAC(manager+) + CSRF, org-scoped via withTenant.
- UI: NONE yet - connecting a channel and sync are API-only (no dashboard screen). Reservations surface only as dashboard widgets.
- Data flow: iCal URL -> POST /v1/channels -> sync -> IcalConnector fetch -> RFC-5545 parse -> normalize -> idempotent upsert (tenant tx) -> audit -> dashboard reads live data.
- Tests: sync (import/update/blocked/error), iCal parse/normalize, RLS isolation.

4. Channel Connectors

Channel	Mechanism	Status
---------	-----------	--------

Generic iCal	RFC-5545 fetch + parse + normalize	[OK] implemented & tested
Airbnb	iCal export URL (host-provided)	[OK] supported / [X] none connected
Booking.com	iCal export URL (extranet)	[OK] supported / [X] none connected

- Which channels are connected: ZERO - no deployment, no host has added a feed.
- Sync frequency: MANUAL only (POST /v1/channels/:id/sync). No scheduler/cron yet - top reservation gap.
- Limitation (by design): iCal carries dates/blocks but NO prices or guest counts, so revenue cannot be derived from it.

5. Dashboard - real vs. sample

Widget	Data source
Upcoming Check-ins	[OK] LIVE reservations (when synced)
Upcoming Check-outs	[OK] LIVE reservations
Occupancy	[OK] LIVE (reservation day-coverage)
Revenue (MTD/ADR/RevPAR)	[~] SAMPLE / estimated (no price source)
Property Health Score	[~] SAMPLE
AI Recommendations	[~] SAMPLE
Recent Leads	[~] SAMPLE
Notification Center	[~] SAMPLE

Property name/type/units are real (from onboarding). The demo banner remains until revenue/reviews/ops modules land; falls back fully to sample when no reservations exist.

6. Smart Inbox [X] not started

- Channels supported: none. No inbox/messaging code exists (verified).
- AI response capabilities: none.
- Intentionally deferred per the guardrail: no inbox until reservation data flows.

7. AI Features

Agent	Status
Property Analyzer (Onboarding Scout)	[OK] built & tested - Claude (claude-opus-4-8) + heuristic fallback
Revenue Manager (dynamic pricing)	[X] not started
Marketing Manager (campaigns/win-back)	[X] not started
Reputation Manager (reviews)	[X] not started

Only the Analyzer exists. The model-router/agent seam (services/ai) is in place to add the others, but none are implemented.

8. Current Phase Completion

- Phase 1 (Public Foundation): 100%.
- Phase 2 (Operational MVP): ~25-30%.
- Done: data spine (Reservation/Channel + RLS), first connector (iCal), live booking widgets, deploy artifacts, CI-verified migrations + isolation.
- Not done: provisioning/deploy, scheduled sync, channel-connect UI, revenue/reviews data,

inbox/pricing/reputation modules.

9. Remaining Work Before First Beta Customer

Must-have (blockers)

- Provision & deploy - Postgres+pgvector, Redis, API+AI (Railway), Web (Vercel); set env; run migrations; confirm /v1/health/ready + live RLS test.
- Scheduled sync worker - cron (e.g. hourly) so reservations refresh automatically; today it's manual.
- Channel-connect UI - a dashboard screen for hosts to paste an iCal URL and sync. The activation moment.
- Real Google OAuth + Resend credentials + a captcha on public analyzer/contact endpoints.

Strongly recommended for a credible beta

- Analyzer prefill from the saved Property (logged-in run).
- A revenue signal so the Revenue widget isn't sample (manual ADR field or PMS/price source) - or clearly label estimates in-product.
- Observability: Sentry + structured logs + sync-failure surfacing on the connection.
- One e2e happy-path test (signup -> onboarding -> connect channel -> dashboard).

10. Biggest Technical Risks

- RLS not yet exercised on a real deploy with a non-owner role. Proven in CI, but if prod connects as table owner, FORCE is bypassed. Mitigation: least-privilege app role (runbook flags it).
- Migrations have only run in CI, never on managed Postgres. Hand-authored SQL + set_config/pgvector may surface provider quirks. Mitigation: staging migrate before prod.
- Manual-only sync + iCal limits. Stale data without a scheduler; iCal lacks price/guest data and UUIDs can shift between providers (duplicate/missed risk). Mitigation: scheduled worker + monitor upsert behavior per provider.
- Cross-origin cookie auth (Vercel web <-> Railway API). SameSite/COOKIE_DOMAIN/CORS misconfig is the likely launch-day bug. Mitigation: shared parent domain; test refresh/CSRF in staging.
- No runtime observability yet. Failures logged server-side but not aggregated/alerting. Mitigation: Sentry + health dashboards before onboarding hosts.
- AI cost/latency unmetered in practice. Router + heuristic fallback exist, but real Anthropic usage/cost per tenant untested at volume. Mitigation: usage logging + caps before scaling beta.

Bottom line

The Phase 2 engineering foundation is solid and honestly de-risked (real RLS test in CI, clean connector, live booking widgets). The gap to a first beta customer is now mostly operational - provision + deploy, add scheduled sync, and ship a channel-connect UI - not architectural. Recommended next build: scheduled sync worker + channel-connect UI.